

Legal Resource-Unit Allocation by Oracle Imitation for Wi-Fi 6 Downlink Scheduling

Author Names and Affiliations to Be Supplied

Abstract—Downlink OFDMA couples queue service with discrete resource-unit constraints. We formulate an instantaneous service-rate oracle over legal Wi-Fi 6 partitions and learn its actions from live access-point queues in ns-3.48. A compact set network and combinatorial decoder produce legal physical allocations. Across six controlled scenarios, the throughput policy remains within 0.02 Mbps of its oracle and improves saturated throughput from 63.99 to 92.12 Mbps against default round-robin. Host decision time averages 824.5 microseconds. Baseline capability differences, transient delay measurements, and age-weighted imitation failures delimit these results; hardware real-time operation remains unverified.

Index Terms—IEEE 802.11ax, OFDMA, resource allocation, imitation learning, ns-3.

I. Introduction

DOWNLINK orthogonal frequency-division multiple access (OFDMA) lets a Wi-Fi 6 access point (AP) transmit to several stations (STAs) using different resource units (RUs). The scheduler must select recipients and place their RUs while accounting for buffered traffic, aggregation, and a common transmission duration. Large RUs can empty short queues quickly, whereas narrow RUs can increase concurrency but prolong a transmission. Legal RU layouts impose more structure than a sum-of-tones constraint.

Optimal and practical 802.11ax allocation algorithms already exist [1], [2]; ns-3-based studies show the importance of MAC overhead and queueing [3]. Reinforcement learning has also been applied to joint user/resource decisions [4]. On commercial Wi-Fi 6 hardware, LOMAS demonstrates latency-driven downlink scheduling [5]. These works motivate a narrower question: can an inexpensive learned decision preserve the service utility of a legal per-opportunity optimizer when both operate inside the same AP MAC?

We study this question through supervised oracle imitation. During data collection, an exact finite-action optimizer labels the AP's live queue at each eligible scheduling opportunity. During evaluation, a set network emits per-STA RU scores and a deterministic decoder selects a legal physical allocation. The contribution is an explicit contract linking the oracle, training labels, and executed RU action, together with an evaluation of its network performance and computation cost. The oracle and decoder share the same partition representation; the actual frame exchange remains in ns-3. The learning architecture builds on established set processing [6], and we do not claim a new general learning paradigm.

The primary result concerns the throughput-trained policy. An age-weighted variant tests the dependence on the label objective and exposes failures of generalization. We retain these failures and evaluate delay only with its finite-window interpretation. The study covers one controlled 20 MHz BSS; it makes no claim of optimal long-run scheduling or device-level real-time operation.

II. Native Queue-State Oracle

A. State and Legal Actions

Consider at most $N = 9$ STAs, one spatial stream, and one best-effort traffic identifier (TID). At a downlink opportunity, the AP observes occupancy, modulation and coding scheme (MCS) m_i , and the first $K_i \leq 64$ BlockAck-eligible queued MPDUs for STA i . Packet p has MAC payload size ℓ_{ip} , encoded MPDU size b_{ip} , and MAC enqueue age d_{ip} . The eligibility check uses ns-3's queue and BlockAck machinery rather than assuming every queued packet can be aggregated immediately.

Let $\mathcal{P}$ contain legal 20 MHz HE RU partitions and $\mathcal{R}(P)$ their physical RU slots. Each RU has 26, 52, 106, or 242 tones. A binary assignment z_{ir} maps STA i to slot r subject to

$$\sum_{r \in \mathcal{R}(P)} z_{ir} \leq 1, \quad \sum_i z_{ir} \leq 1, \quad 1 \leq \sum_{i,r} z_{ir} \leq M \leq 9. \quad (1)$$

Empty or BlockAck-ineligible stations cannot be selected. Unused RUs are allowed. An action $a = (P, z)$ therefore specifies both the partition and an injective assignment to physical slots. Under the present channel model, the utility of an RU depends on its size but not its frequency position. Partitions with identical RU-size multisets are consequently represented by one canonical physical layout. This equivalence would not hold with frequency-selective channel quality or interference.

B. Instantaneous Service Objective

For a given partition and HE-SIG-B MCS class c , the implementation computes the header duration $h(P, c)$ using ns-3's PHY timing functions. Here $c = \min(5, \min_{i \text{ selected}} m_i)$. The header estimate accounts for all slots in the chosen layout. For each eligible pair (i, r) , let $k_{ir}(P, c)$ be the longest queued prefix satisfying the negotiated A-MPDU byte limit, the BlockAck window, and the 5.484 ms HE-MU duration limit. Delimiters and padding enter the encoded aggregate size. ns-3's payload-duration function returns its transmission time $t_{ir}(P, c)$.

The main label maximizes instantaneous payload service per estimated exchange time. To examine objective dependence, define

$$W_{ir}(\lambda) = \sum_{p=1}^{k_{ir}} \ell_{ip} \left(1 + \lambda \frac{d_{ip}}{d_0} \right), \quad (2)$$

$$J_\lambda(a; s) = \frac{8 \sum_{i,r} z_{ir} W_{ir}(\lambda)}{T_{oh} + h(P, c) + \max_{i,r: z_{ir}=1} t_{ir}}, \quad (3)$$

$$a_\lambda^*(s) \in \arg \max_{a \text{ satisfying (1)}} J_\lambda(a; s). \quad (4)$$

We use $T_{oh} = 100 \mu s$, a fixed approximation to contention/control overhead, and $d_0 = 100$ ms. The throughput oracle O-T uses $\lambda = 0$; the age-weighted oracle O-A uses $\lambda = 2$. Neither score includes a packet-error probability. Positive λ increases the reward for serving older packets; it is a delay-pressure surrogate, not an objective for future mean packet delay. Equations (2)–(4) optimize RU allocation under a fixed longest-prefix aggregation rule; they do not jointly optimize all shorter aggregation depths. Final MAC admission can further restrict the predicted service.

C. Exact Finite-Action Search

For every (P, c) , enumerate the distinct feasible pair durations $\tau \in \{t_{ir}(P, c)\}$. Retain pairs with $t_{ir} \leq \tau$ and $\min(5, m_i) \geq c$. A dynamic program (DP) processes the physical RU slots. Let $D_j(S)$ be the largest sum of rewards after j slots with exactly the STA subset S assigned. Starting from $D_0(\emptyset) = 0$ and $D_0(S) = -\infty$ otherwise, its recurrence is

$$D_{j+1}(S) = \max \left\{ D_j(S), \max_{i \in S} [D_j(S \setminus \{i\}) + W_{ir_{j+1}}] \right\}, \quad (5)$$

where the inner maximum admits only retained pairs and $|S| \leq M$. The first term leaves an RU unused. Terminal sets must be nonempty and include a STA realizing class c . Predecessors recover the assignment; we evaluate its actual maximum pair duration in (3) and retain the best action over all thresholds and layouts.

Exactness under the stated model: an optimal action has a maximum pair duration equal to an enumerated threshold τ^* . At its partition, class, and terminal STA set, the DP finds a numerator at least as large while the maximum pair duration is no greater than τ^* . Its objective is therefore at least that of the optimum. Since every reconstructed action is feasible, equality follows. This establishes exactness of the finite surrogate, not an upper bound on cumulative delivered traffic.

For P_0 canonical layouts, $C \leq 6$ header classes, L thresholds, and $R \leq 9$ slots, the DP core costs $O(P_0 CLRN 2^N)$ operations. The current implementation scans up to $K = 64$ prefix options inside pair lookups, adding a factor K to a conservative implementation bound. Predecessors require $O(R 2^N)$ storage per threshold. Enumerating this search for data collection avoids requiring it at each learned-policy decision.

III. Learning and Native Execution

A. Features and Training

Each STA has 39 normalized inputs: eligibility, MCS, 20 MHz PHY rate, queue packet and byte occupancy, head-of-line and mean observed-prefix age, and size/age pairs for the first 16 eligible packets. Absent positions are zero. Packet counts use $q/(q+64)$; byte occupancy is clipped at 64×1600 ; packet size and age are clipped at 1600 bytes and 2000 ms. The learned execution path reads at most 16 packets per STA. Oracle snapshots preserve up to 64 packets; their mean-prefix-age feature therefore uses a potentially longer prefix. This remaining feature mismatch is retained in the released corpus rather than retrospectively altering the experiment.

A shared encoder maps row x_i to u_i . Masked mean and maximum pooling over eligible STAs produce global context

$$g = \rho([\text{mean}_i u_i, \max_i u_i]), \quad v_i = \psi([u_i, g]). \quad (6)$$

A linear head gives logits $v_{i,q}$ for $q \in \{0, 26, 52, 106, 242\}$, where zero denotes no service. Encoder and decoder each contain two width-64 ReLU layers; the context encoder has one. Shared weights and symmetric pooling ensure that permuting input rows permutes the logits. Decoder ties can still select a different equivalent physical action after renumbering.

O-T and O-A generate separate corpora of 11,297 and 11,149 labeled states from 24 trajectories each. The deterministic scenario generator combines 3, 6, and 9 STAs; MCS 0, 3, 6, 9, and 11; four datagram sizes (300, 800, 1200, and 1500 bytes); five load levels; and three skews. Each collection run starts traffic at 1 s, uses 1 s warm-up and 0.3 s measurement, and stops 0.1 s later. Labels are written throughout eligible oracle operation, including warm-up. Empty actions do not create samples.

Four complete trajectories form validation, leaving 20 for training. The throughput split contains 10,267 training and 1,030 validation states. We train for 40 epochs with AdamW (learning rate 10^{-3} , weight decay 10^{-4}), batch size 256, dropout 0.05, gradient clipping at 5, and random STA permutations. Class weights are proportional to the inverse square root of training frequency. Cross entropy uses only eligible rows; checkpoint selection maximizes validation exact-action accuracy, then selection F1. No evaluation trajectory enters this split. Only the RU head supplies scheduling decisions; unused legacy heads remain in the checkpoint format.

B. Legal Decoding and MAC Integration

Independent argmax classes need not form a legal partition. The decoder solves

$$\hat{a} = \arg \max_{(P,z) \text{ satisfying (1)}} \sum_{i,r} z_{ir} (v_{i,\text{type}(r)} - v_{i,0}). \quad (7)$$

The none-class subtraction accounts for leaving a STA unserved. Recurrence (5), now using logit differences, solves (7) in $O(P_0 R N 2^N)$ time without PHY evaluation

TABLE I
Native evaluation scenarios; all datagrams are 1,000 bytes

Scenario	N	MCS	b	η	Total Mbps
Sparse	9	8	0.7	0.2	6.3
Normal	9	8	5.5	0.35	49.5
Saturated	9	8	16	0.65	144.0
Low MCS	9	2	5	0.5	45.0
4 STAs	4	8	24	0.3	88.8
7 STAs	7	5	12	0.55	77.4

or threshold enumeration. Any nonempty decoded action obeys the structural RU constraints by construction. This property guarantees legal placement, not successful MAC admission or reception. Empty actions use the inherited scheduling path.

The exported binary occupies 111,268 bytes and uses a C++ runtime without a tensor-library dependency. We extend RrMultiUserScheduler, install the partition code and each physical HeRu::RuSpec in the WifiTxVector, and extend mixed-layout HE-SIG-B handling. ns-3 then performs sequence assignment, actual aggregation, contention, acknowledgments, and retries. Unit checks cover simple oracle decisions, age weighting, and legal projection; the artifact includes the upstream RU-allocation, OFDMA, aggregation, queue, and duration test commands. No result here relies on substituting predicted service for received traffic.

IV. Native ns-3 Evaluation

A. Protocol and Baselines

The network has a single AP, stationary STAs on a 2 m circle, 5 GHz 20 MHz HE operation, 800 ns guard interval, one spatial stream, and a common fixed MCS per run. We use SpectrumWifiPhy, log-distance path loss, UDP periodic arrivals, no A-MSDU, a 64-MPDU BlockAck window, and no uplink OFDMA. Other settings retain the packaged ns-3 defaults. For STA index i starting at zero, offered rate is $b[1 + \eta((i \bmod 3) - 1)]$ Mbps. Table I gives the six evaluation cases. The 1,000-byte datagram size is absent from training. These held-out configurations test interpolation in a controlled family, not unrestricted traffic generalization.

RR4 is ns-3's credit round-robin scheduler with its default four-STA limit. Max-Q sorts eligible destinations by backlog and retains the stock RU allocation path, with a nine-STA limit. O-T/O-A and their learned policies L-T/L-A allow nine STAs and legal mixed partitions. Thus RR4 is an operational default baseline; the study does not isolate learning from the effect of different station limits and allocation capabilities. Max-Q controls the station limit, but not the mixed-layout action space.

Each of six policies is run on each scenario with ten paired RNG run numbers (920000–920009), giving 360 runs. Traffic starts at 1 s; measurement covers [3, 4] s after 2 s of traffic warm-up. The simulation ends at 4.1 s. Throughput counts UDP bytes received in the measurement window divided by one second, including

TABLE II
Received UDP throughput (Mbps), ten paired runs

Scenario	RR4	Max-Q	O-T	L-T	O-A	L-A
Sparse	6.31	6.31	6.31	6.31	6.31	6.31
Normal	49.83	49.49	49.49	49.48	49.50	49.49
Saturated	63.99	65.90	92.13	92.12	83.32	83.75
Low MCS	17.35	16.32	23.04	23.04	23.04	23.04
4 STAs	69.51	69.51	88.79	88.79	88.80	79.06
7 STAs	43.02	47.56	61.60	61.59	57.42	48.88

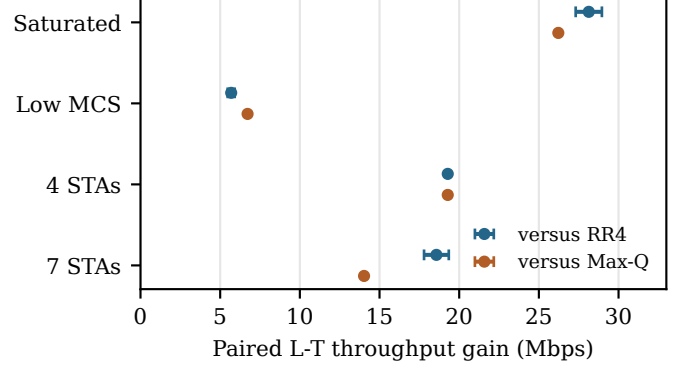


Fig. 1. L-T gains in the four overloaded cases; bars are paired 95% confidence intervals over ten runs. Baseline capabilities differ as stated in Section IV-A.

the sequence/timestamp header. Delay is measured from UDP generation for those received packets. It excludes dropped or still queued packets and is conditioned on reception time. Paired confidence intervals use Student's t with nine degrees of freedom; the six-scenario analysis is exploratory without multiplicity correction.

B. Throughput and Objective Dependence

Table II reports all six policies. L-T obtains 92.12 Mbps versus RR4's 63.99 Mbps at saturation, a 44.0% gain, and exceeds Max-Q by 26.22 Mbps. The paired gains over RR4 are 28.13 ± 0.82 Mbps at saturation, 5.69 ± 0.22 Mbps at low MCS, 19.28 ± 0.03 Mbps with four STAs, and 18.57 ± 0.78 Mbps with seven STAs (Fig. 1). These effects are repeatable for the tested configuration. The four-STA result cannot be attributed to a higher recipient cap, since both policies can address all four STAs; it still combines allocation and policy differences.

L-T is within 0.02 Mbps of O-T in every case. This is a comparison of closed-loop throughput means, not a bound on instantaneous objective regret. Raw exact-action accuracy is only 11.46%, and accuracy on eligible STA rows is 62.22%. Such discrepancies can arise from equivalent actions and the constraint decoder; their individual contributions have not been isolated. A direct greedy decoder and objective-regret evaluation are needed before attributing the gain solely to learning.

At sparse and normal load the policies are offer-limited. Normal RR4's 49.83 Mbps slightly exceeds the 49.50 Mbps offered rate because the measurement can include packets generated during warm-up. The same boundary effect

TABLE III
Delay of packets received in the measurement window (ms)

Scenario	RR4 mean	L-T mean	RR4 p99	L-T p99
Sparse	0.56	0.87	1.97	0.88
Normal	10.91	3.70	30.74	5.24
Saturated	562.47	518.26	929.88	922.45
Low MCS	1007.83	840.41	2012.90	1652.38
4 STAs	419.34	7.16	1207.71	16.02
7 STAs	782.48	356.67	1755.43	971.11

TABLE IV
Serial host timing: means across 18 runs (μ s)

Component	Mean	Mean of run p99s
State/features	163.7	195.4
Inference	175.3	192.8
Projection	483.0	640.9
Decision hook	824.5	1016.0

permits small deviations above offered rate in other cases. Short measurement windows therefore require particular caution near the transition to saturation.

L-A does not uniformly reproduce O-A: it loses 9.74 Mbps with four STAs and 8.54 Mbps with seven STAs, despite higher raw exact-action accuracy (34.39%). At saturation it obtains 83.75 Mbps and Jain’s index 0.926, compared with L-T’s 92.12 Mbps and 0.907. These indices describe equality of received per-STA throughput; offered rates are unequal. The age weight changes the operating point but does not establish an optimal fairness or delay tradeoff. Both training objectives and all underlying runs are included in the artifact.

C. Interpreting Packet Delay

Table III shows lower received-packet mean delay for L-T in the four overloaded cases, but these are transient measurements. At saturation, offered load is 144 Mbps and L-T serves about 64% of that rate. Neither policy can sustain all arrivals under this configuration. With four STAs, L-T nearly serves the entire 88.8 Mbps offered load while RR4 serves about 78%; the large delay difference primarily accompanies this service-capacity gap. The ratio of received throughput to offered rate is not a measured packet-cohort delivery probability.

Finite buffers, queue expiry, and the observation boundary further affect which packets contribute to delay. We therefore claim neither queue stability nor a steady-state delay improvement under overload. At sparse load L-T’s mean delay rises from RR4’s 0.56 to 0.87 ms. Stable-load sweeps with explicit drop, backlog, and packet-cohort accounting are required for a general latency claim. In particular, adding age to (2) has not solved average-delay optimization.

D. Computation Cost

An independent serial campaign uses three RNG runs per scenario (940000–940002). The host is an Intel Xeon

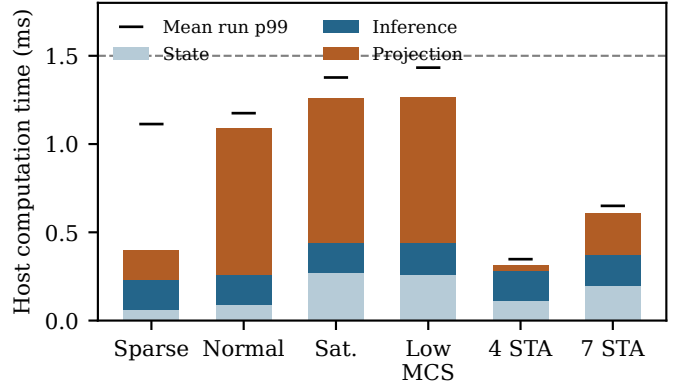


Fig. 2. Serial L-T host cost by scenario. Stacks show mean component times; black ticks show means of run-level decision p99s. The dashed line is the chosen 1.5 ms budget. Minor hook bookkeeping is outside the stacks.

Gold 5222 at nominal 3.80 GHz; GCC 14.2.0 builds the packaged ns-3 default profile with `-Os -g -DNDEBUG` and ns-3 assertions/logging enabled. `std::chrono::steady_clock` measures state extraction, model inference, projection, and the complete selection hook. The hook includes assignment bookkeeping and station ordering, but subsequent MAC frame construction and hardware submission are outside the measured interval. Timers cover all instrumented opportunities, including warm-up and drain.

The overall mean inference and hook costs are 175.3 and 824.5 μ s, respectively (Table IV). Projection dominates at nine STAs (Fig. 2); a small network alone does not ensure a small complete scheduling cost. The reported 1016.0 μ s is the average of 18 within-run p99s, not a pooled p99. The largest observed hook time is 3261.6 μ s. At an illustrative 1.5 ms budget, the mean run miss fraction is 0.12%; this is neither a deadline derived from the MAC protocol nor a worst-case guarantee. Empty-action fallback averages 4.94 occurrences per run, including initial setup; fallbacks are distinct from deadline misses.

Computation time is not charged to ns-3 simulated time. Consequently, these measurements establish host software cost, while network results assume scheduling decisions are available immediately. A deployment claim requires a justified decision deadline, evaluation with computation delay and stale state, and measurements on the intended AP processor. The exponential STA-mask decoder also limits extrapolation beyond nine STAs.

V. Conclusion

An oracle and learned decoder sharing legal physical RU actions provide a reproducible route from live AP queues to native downlink OFDMA scheduling. In six controlled ns-3 cases, the throughput policy closely tracks its oracle and improves overloaded throughput over default RR. The age-weighted model’s failures, unequal baseline capabilities, and conditional delay statistics constrain the interpretation. Host timing identifies legal projection as a major cost. Longer runs, matched mixed-RU heuris-

tics, heterogeneous time-varying channels, and interference experiments remain necessary to assess robustness. The accompanying source artifact provides the simulator snapshot, data, models, and commands for regenerating every reported table and figure.

References

- [1] K. Wang and K. Psounis, “Efficient scheduling and resource allocation in 802.11ax multi-user transmissions,” *Computer Communications*, vol. 152, pp. 171–186, 2020.
- [2] A. Dutta, N. Gupta, S. Das, and M. Maity, “MMRU-ALLOC: An optimal resource allocation framework for OFDMA in IEEE 802.11ax,” in *Proc. IEEE PIMRC*, 2020, pp. 1–7.
- [3] D. Magrin, S. Avallone, S. Roy, and M. Zorzi, “Performance evaluation of 802.11ax OFDMA through theoretical analysis and simulations,” *IEEE Trans. Wireless Commun.*, vol. 22, no. 8, pp. 5070–5083, 2023.
- [4] H. Noh, H. Lee, and H. J. Yang, “Joint optimization on uplink OFDMA and MU-MIMO for IEEE 802.11ax: Deep hierarchical reinforcement learning approach,” *IEEE Commun. Lett.*, vol. 28, no. 8, pp. 1800–1804, 2024.
- [5] Y.-A. Tai, Y.-W. Liu, P.-K. Kao, T.-Y. Lee, C.-I. Hu, and C.-Y. Li, “LOMAS: Latency-driven OFDMA scheduling design in Wi-Fi 6 networks,” in *Proc. IEEE ICNP*, 2025.
- [6] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. J. Smola, “Deep sets,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3391–3401.